The Journal of Open Source Software

HighFidelityEphemerisModel.jl: A Julia package for high-fidelity astrodynamics

Yuri Shimane [1¶] **and Derek Phung** [1]

1 Department of Mechanical and Aerospace Engineering, University of California, Irvine, United States ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

HighFidelityEphemerisModel.jl is an open-source Julia (Bezanson et al., 2017) package for modeling high-fidelity translational dynamics of spacecraft. The package provides equations of motion that account for the gravitational influence of multiple celestial bodies using ephemeris data from the Jet Propulsion Laboratory's SPICE ecosystem (Acton, 1996), together with configurable perturbations from a spherical harmonics model of gravitational irregularities, solar radiation pressure, and atmospheric drag. It is designed to be fully compatible with Julia's OrdinaryDiffEq.jl ecosystem (Rackauckas & Nie, 2017).

The package supports trajectory propagation in Cartesian coordinates and Gauss variational equations formulated in modified equinoctial, classical Keplerian, and ordinary equinoctial elements. Additional capabilities include state-transition matrix propagation, Jacobian and Hessian evaluation, event-condition utilities, and export of propagated trajectories to SPICE spacecraft kernels. Ephemeris and reference-frame evaluations are supported through the SPICE kernels ecosystem. These capabilities enable researchers to incorporate high-fidelity spacecraft dynamics into trajectory analysis, sensitivity analysis, and optimization workflows, while also leveraging Julia's scientific computing libraries.

Statement of need

Accurate spacecraft trajectory propagation is essential for mission design, trajectory optimization, and dynamical analysis. Simplified models often provide valuable insight into spacecraft motion, but are insufficient in designing flyable nominal trajectories or in assessing realistic performance of guidance, navigation, and control (GNC) algorithms. While mature numerical integration libraries provide suitable algorithms for solving general initial value problems (IVP), these must be paired with efficient implementation of these high-fidelity dynamics models, often involving not only the equations of motion but also the variational equations that yield sensitivity information, commonly used by workflows in Astrodynamics involving numerical optimization, filtering, and sensitivity analysis. This software addresses this need by providing configurable, ephemeris-based spacecraft dynamics within Julia's scientific computing ecosystem. By separating the equations of motion from the numerical integration algorithms, the package enables researchers and mission analysis engineers to combine high-fidelity dynamical models with existing differential-equation solvers and optimization tools. Figure \ref{fig:hfem-hero} demonstrates some of the capabilities of HighFidelityEphemerisModel.jl.

DRAFT

The Journal of Open Source Software

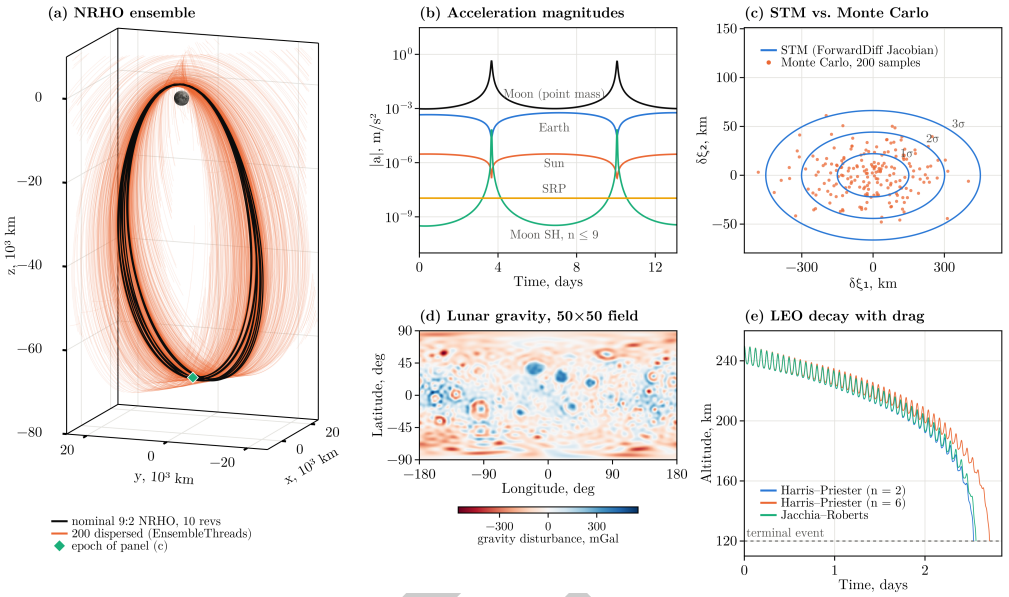


Figure 1: Demonstration of `HighFidelityEphemerisModel.jl` capabilities.

State of the field

Several software packages provide high-fidelity spacecraft dynamics and trajectory propagation capabilities in languages other than Julia. Orekit (Maisonobe et al., 2010) is a Java-based flight dynamics library offering comprehensive orbital propagation, reference-frame transformations, and mission analysis functionality. NASA's General Mission Analysis Tool (GMAT) (Hughes et al., 2014) provides an integrated environment for trajectory design, optimization, and navigation through graphical, scripting, and programmatic interfaces. Basilisk (Kenneally et al., 2020) is a modular simulation framework that combines spacecraft dynamics with guidance, navigation, and control models, exposed as a python package. Nyx (Nyx Space, 2026) provides a Rust-based astrodynamics toolkit for trajectory propagation and orbit determination, while Brahe (Eddy & Kochenderfer, 2026) offers a composable astrodynamics library with Rust and Python interfaces, particularly for satellite dynamics and mission operations.

While these tools offer extensive capabilities, `HighFidelityEphemerisModel.jl` addresses a more focused requirement: providing configurable, ephemeris-based equations of motion that can be directly incorporated into Julia's scientific computing workflows. Rather than providing a comprehensive mission analysis toolkit or implementing separate numerical integration algorithms, `HighFidelityEphemerisModel.jl` exposes reusable dynamical models compatible with `OrdinaryDiffEq.jl`. This enables researchers to combine efficient, high-fidelity force models with numerical integration, sensitivity analysis, and trajectory optimization methods within the same programming environment.

Within the Julia ecosystem, there are a number of mature spacecraft-related packages, albeit focused on different scopes. `SatelliteToolbox.jl` offers satellite-analysis tools, including analytical orbit propagators and terrestrial reference-frame utilities (JuliaSpace contributors, 2026). `GeneralAstrodynamics.jl` provides common astrodynamics calculations and solvers for Keplerian, Lambert, and halo-orbit problems (JuliaAstro contributors, 2026). `AstroModels.jl` develops reusable gravitational and other astrodynamical force models (Julia Space Mission Design contributors, 2026a). `HighFidelityEphemerisModel.jl` complements these projects by packaging ephemeris-based spacecraft equations of motion, multiple ephemeris backends, and Cartesian and Gauss variational formulations for direct use with Julia differential-equation solvers.

DRAFT

The Journal of Open Source Software

Software design

Dynamics models

`HighFidelityEphemerisModel.jl` formulates spacecraft motion relative to a central celestial body, such as a planet, a moon, or the Sun, with additional, configurable perturbations due to gravitational irregularities of the central body modeled via spherical harmonics, third-body point-mass perturbations by other celestial bodies, solar radiation pressure, and atmospheric drag. The package supports propagation of the spacecraft state in terms of its inertial position and velocity vectors, as well as in terms of orbital elements throug Gauss variational equations. Their implementation support rescaling the dynamics via canonical scales for improved numerical stability, selected by a user defined distance unit DU, and an automatically scaled velocity unit $\mathrm{VU} = \sqrt{\mu/\mathrm{DU}}$, where μ is the central body's gravitational parameter.

Modular implementation

A central architectural decision is the separation of dynamical models from ephemeris evaluation and numerical integration. `HighFidelityEphemerisModel.jl` provides two parameter backends: `SpiceParameters`, which queries NAIF SPICE directly (Acton, 1996); and `EphemeridesParameters`, which uses the Julia-native `Ephemerides.jl` and `FrameTransformations.jl` libraries (Julia Space Mission Design contributors, 2026b, 2026c). Julia's multiple dispatch allows the same high-level equations-of-motion functions, `eom_Nbody!` and `eom_NbodySH!`, to operate with these different parameter types. Direct SPICE evaluation provides an established reference implementation, while the Julia-native backend supports automatic differentiation and tested multithreaded propagation workflows without relying on direct CSPICE calls during each force evaluation.

Numerical integration is delegated to `OrdinaryDiffEq.jl` (Rackauckas & Nie, 2017). This self-contained example propagates a Moon-centered state with Earth and Sun point-mass perturbations.

The package also provides automatic-differentiation-based Jacobian and Hessian calculations through `ForwardDiff.jl` (Revels et al., 2016), together with state-transition matrix propagation for sensitivity analysis. Propagated trajectories can be exported to SPICE type-13 ephemeris kernels, with utilities supporting multiple trajectory segments and maneuver metadata. Together, these capabilities facilitate the integration of high-fidelity spacecraft dynamics into trajectory optimization, GNC algorithm designs, and reproducible mission-analysis workflows.

Figure \ref{fig:hfem-architecture} summarizes these interfaces.

DRAFT

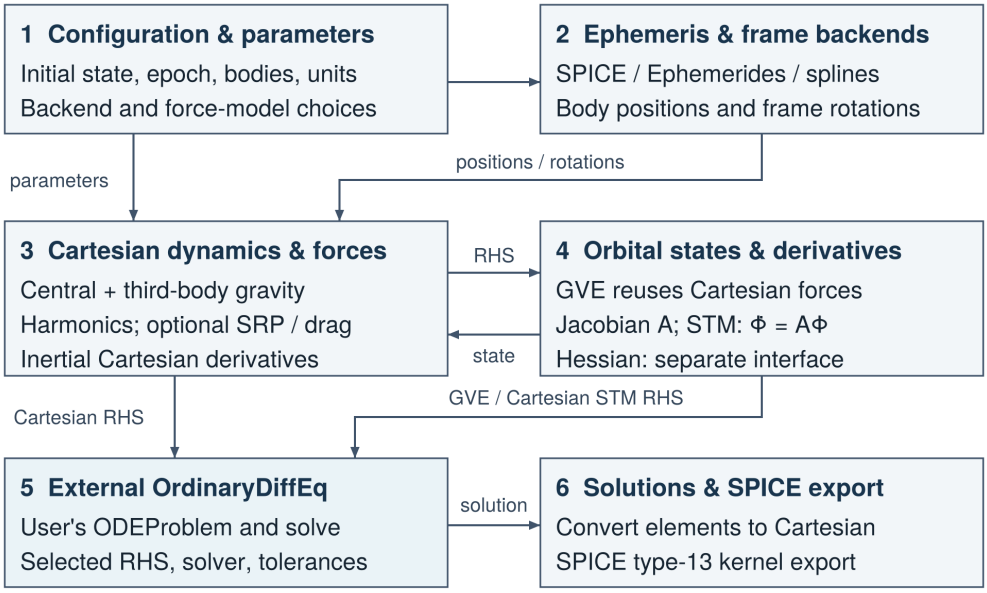


Figure 2: `HighFidelityEphemerisModel.jl` architecture. Cartesian forces support direct propagation and Gauss variational equations (GVE). Jacobians drive Cartesian state-transition matrix (STM) evolution; Hessians are separate. Element solutions require Cartesian conversion before SPICE export.

Validation

The package has been evaluated through numerical comparisons with established astrodynamics libraries, including GMAT, Orekit, Nyx, and Basilisk. Three representative cases encompass a seven-day near-rectilinear halo orbit (H7), a low lunar orbit (LLO-MESH4), and a low Earth orbit (LEO-EMSH4). Each case includes third-body gravitational perturbations and spherical harmonics terms up to degree/order-four. Residuals use documented `HighFidelityEphemerisModel.jl` SPICE trajectories, with tool-specific numerical mirrors for Nyx and LLO Orekit; they measure agreement, not absolute accuracy. These comparisons provide quantitative evidence of numerical consistency across different propagation frameworks while accounting for differences in integrators, ephemeris evaluation, and model implementation.

Table \ref{tbl:validation-cases}, Figure \ref{fig:validation-position}, and Table \ref{tbl:position-maxima} summarize these comparisons. GMAT H7 uses shared MOON_PA orientation with an inertial-frame correction. GMAT LLO/LEO endpoint offsets from nominal grids reach 2.70 microseconds. Basilisk uses 1 s communication/task updates with adaptive integration; Orekit uses shared frame orientations.

Table 1: Validation cases; frames specify harmonic evaluation axes. LLO/LEO each span one period. \label{tbl:validation-cases}

Case	Central body	Third bodies	Spherical harmonics frame	Integration time
H7	Moon	Earth + Sun	`MOON\_PA`	7 d
LLO-MESH4	Moon	Earth + Sun	`MOON\_PA`	2.064 h
LEO-EMSH4	Earth	Moon + Sun	`IAU\_EARTH`	1.978 h

DRAFT

Table 2: Maximum sampled position residuals against documented `HighFidelityEphemerisModel.jl` with `SpiceParameters` or numerical mirrors, not absolute trajectory errors. First row uses `HighFidelityEphemerisModel.jl` with `EphemeridesParameters`.\label{tbl:position-maxima}

Implementation	H7 [mm]	LLO-MESH4 [mm]	LEO-EMSH4 [mm]
`EphemeridesParameters`	0.337568	0.0349237	0.148159
GMAT	136.282	3.07436	32.1019
Orekit	5.31215	1.51733e-05	0.000104418
Nyx.	0.00121614	0.221886	1.02998
Basilisk	0.347638	0.0262097	101.602

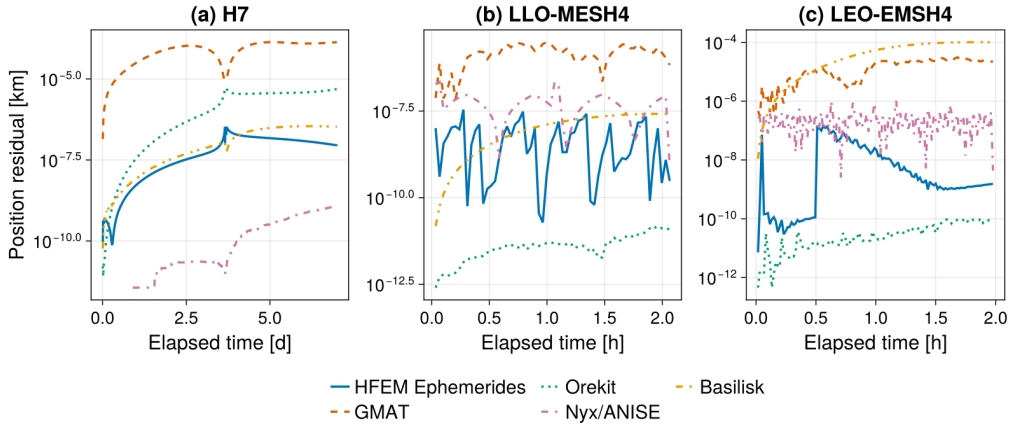


Figure 3: Position residuals against `HighFidelityEphemerisModel.jl` SPICE trajectories or tool-specific numerical mirrors, not absolute trajectory accuracy. Initialization ($t = 0$) is omitted; later stored zeros appear as logarithmic gaps without flooring. Output sampling differs from integration steps.

Additional benchmarks evaluate `HighFidelityEphemerisModel.jl`'s alternative ephemeris backends. The Julia-native `Ephemerides.jl` backend reduced propagation time relative to direct SPICE evaluation from 0.2473 to 0.1236 s for a point-mass near-rectilinear halo orbit case, and from 0.8677 to 0.7031 s with lunar spherical harmonics. These timings are configuration-specific, not universal speedups.

Table \ref{tbl:warm-performance} reports end-to-end propagation at `HighFidelityEphemerisModel.jl` source commit `2aca3a70ffffad4e61db28f68c8c9d6d7630f1e9` (Julia 1.10.11). For each case/backend, an initial untimed solve excludes the initial Julia compilation run; a separate allocation-measurement solve also precedes the ten timed repetitions. No compilation time is subtracted from measured runs. Timings include ODE construction, Vern8 propagation and in-memory output with initialized backends; backend setup and serialization are excluded.

Table 3: Median propagation (ten repetitions, one Julia thread). Ratio: Ephemerides/SPICE. P3 uses point masses; H7 adds lunar 4x4. \label{tbl:warm-performance}

Case	SPICE [s]	Ephemerides [s]	Time ratio
P3	0.2472955	0.1236207	0.4999
H7	0.8677467	0.7031303	0.8103

The package is distributed through Julia's package registry and provides documented examples, automated tests, and continuous integration to support reproducible research and further software development.

DRAFT

Example

The following code provides a short example solving an IVP for an initial state in a Moon-centered J2000 inertial frame over the course of 6 hours. The dynamics model accounts for third-body perturbations of the Earth and the Sun.

```julia
using HighFidelityEphemerisModel, OrdinaryDiffEq, SPICE

furnsh(joinpath(ENV["SPICE"], "lsk", "naif0012.tls"))
furnsh(joinpath(ENV["SPICE"], "spk", "de440.bsp"))
furnsh(joinpath(ENV["SPICE"], "pck", "gm_de440.tpc"))

ids = ["301", "399", "10"]                    # Moon first, then Earth and Sun
GMs = [bodvrd(id, "GM", 1)[1] for id in ids]  # gravitational parameters, km^3/s^2
et0 = str2et("2030-01-05T00:00:00")           # initial epoch, UTC
DU = 1e5                                      # canonical scale, km
params = SpiceParameters(et0, DU, GMs, ids, "J2000", "NONE")

x0 = [1.05, 0.0, 0.3, 0.5, 1.0, 0.0]          # canonical units DU, DU/TU
tspan = (0.0, 6 * 3600 / params.TU)           # canonical unit TU
prob = ODEProblem(eom_Nbody!, x0, tspan, params)
sol = solve(prob, Vern8(); reltol=1e-12, abstol=1e-12)
@assert sol.t[end] == tspan[2] && all(u -> all(isfinite, u), sol.u)
kclear()
```

Research impact

`HighFidelityEphemerisModel.jl` has supported several research on high-fidelity trajectory design and GNC in cisluar space (Phung & Shimane, 2026; Shimane, Elango, et al., 2026; Shimane, Isaji, et al., 2026). This application demonstrates the integration of ephemeris-based spacecraft dynamics, sensitivity analysis, and trajectory propagation into larger numerical optimization and mission-analysis workflows.

AI usage disclosure

AI was used to help develop the code and to draft and edit parts of this manuscript. All text produced with AI assistance was reviewed, revised, and verified by the authors before inclusion in the code or paper.

Acknowledgements

The authors acknowledge the developers and maintainers of the Julia scientific computing ecosystem and NASA's Navigation and Ancillary Information Facility (NAIF), whose software and documentation provide important foundations for this work.

References

Acton, Jr., Charles H. (1996). Ancillary data services of NASA's navigation and ancillary information facility. *Planetary and Space Science*, *44*(1), 65–70. https://doi.org/10.1016/0032-0633(95)00107-7

Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM Review*, *59*(1), 65–98. https://doi.org/10.1137/141000671

DRAFT

Eddy, D., & Kochenderfer, M. J. (2026). *Brahe: A modern astrodynamics library for research and engineering applications*. https://arxiv.org/abs/2601.06452

Hughes, S. P., Qureshi, R. H., Cooley, D. S., & Parker, J. S. (2014). Verification and validation of the General Mission Analysis Tool (GMAT). *AIAA/AAS Astrodynamics Specialist Conference*, 4151.

Julia Space Mission Design contributors. (2026a). *AstroModels.jl: Astrodynamical force models*. Software repository. https://github.com/JuliaSpaceMissionDesign/AstroModels.jl

Julia Space Mission Design contributors. (2026b). *Ephemerides.jl: A modern binary ephemeris reader for Julia*. Software repository. https://github.com/JuliaSpaceMissionDesign/Ephemerides.jl

Julia Space Mission Design contributors. (2026c). *FrameTransformations.jl: Reference-frame transformations in Julia*. Software repository. https://github.com/JuliaSpaceMissionDesign/FrameTransformations.jl

JuliaAstro contributors. (2026). *GeneralAstrodynamics.jl: Astrodynamics calculations and solvers in Julia*. Software repository. https://github.com/JuliaAstro/GeneralAstrodynamics.jl

JuliaSpace contributors. (2026). *SatelliteToolbox.jl: A toolbox for satellite analysis in Julia*. Software repository. https://github.com/JuliaSpace/SatelliteToolbox.jl

Kenneally, P. W., Piggott, S., & Schaub, H. (2020). Basilisk: A flexible, scalable and modular astrodynamics simulation framework. *Journal of Aerospace Information Systems*, *17*(9), 496–507. https://doi.org/10.2514/1.I010762

Maisonobe, L., Parraud, P., & Vehil, R. (2010). Orekit: An open source library for operational flight dynamics applications. *Proceedings of the 4th International Conference on Astrodynamics Tools and Techniques (ICATT)*.

Nyx Space. (2026). *Nyx: An open-source astrodynamics toolkit*. Software repository. https://github.com/nyx-space/nyx

Phung, D., & Shimane, Y. (2026). *High-fidelity baseline design and station keeping analyses for earth–moon vertical orbits*. https://arxiv.org/abs/2610.10803

Rackauckas, C., & Nie, Q. (2017). DifferentialEquations.jl: A performant and feature-rich ecosystem for solving differential equations in Julia. *Journal of Open Research Software*, *5*(1), 15. https://doi.org/10.5334/jors.151

Revels, J., Lubin, M., & Papamarkou, T. (2016). *Forward-mode automatic differentiation in Julia*. https://arxiv.org/abs/1607.07892

Shimane, Y., Elango, P., & Weiss, A. (2026). *Optimization-based formation flight on libration point orbits*. https://arxiv.org/abs/2603.09183

Shimane, Y., Isaji, M., Weiss, A., & Di Cairano, S. (2026). Scenario-based model predictive control for station keeping on near-rectilinear halo orbit. *IFAC World Congress*.

DRAFT